



TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Problemas NP-Completos

15 de junio de 2026

Ignacio Malmierca
113695

Nadeska Millan
110779

Introduction

En este trabajo se aborda el estudio del problema *Hitting Set*, un problema clásico de optimización combinatoria que pertenece a la clase de problemas NP-Completo. El objetivo consiste en encontrar un subconjunto mínimo de elementos que interseque a cada uno de los subconjuntos dados de una colección, garantizando que todos ellos queden cubiertos.

A partir de la situación planteada en la consigna, donde se busca seleccionar la menor cantidad posible de jugadores que satisfagan los deseos de distintos medios de comunicación, se modela formalmente el problema como una instancia de Hitting Set. Este modelado permite analizar el problema desde una perspectiva teórica y aplicar herramientas propias del diseño y análisis de algoritmos.

En primer lugar, se estudia la versión de decisión del problema con el fin de demostrar que pertenece a la clase NP y que es NP-Completo mediante una reducción polinomial desde un problema conocido. Luego, se desarrolla un algoritmo exacto basado en la técnica de *Backtracking*, capaz de encontrar una solución óptima explorando sistemáticamente el espacio de soluciones posibles y aplicando podas para reducir la cantidad de casos analizados.

Finalmente, se generan distintos conjuntos de datos para validar la correctitud de la implementación y se realizan mediciones experimentales de tiempo de ejecución. A partir de estos resultados se analiza el comportamiento del algoritmo frente al crecimiento del tamaño de las instancias, contrastando los resultados obtenidos con la complejidad teórica esperada para un problema NP-Completo.

1. Demostración de que Hitting Set pertenece a NP

Consideremos la versión de decisión del problema *Hitting Set*:

Dado un conjunto finito A , una colección de subconjuntos $B_1, B_2, \dots, B_m \subseteq A$, y un entero positivo k , determinar si existe un subconjunto $C \subseteq A$ tal que $|C| \leq k$ y $C \cap B_i \neq \emptyset$ para todo $1 \leq i \leq m$.

Para demostrar que el problema pertenece a la clase NP, debemos exhibir un certificado de tamaño polinomial y un algoritmo verificador que pueda comprobar la validez de dicho certificado en tiempo polinomial.

Como certificado, consideramos un subconjunto $C \subseteq A$ propuesto como solución. Este certificado tiene tamaño a lo sumo $n = |A|$, por lo que su representación es polinomial respecto del tamaño de la entrada.

El procedimiento de verificación consiste en:

1. Verificar que $|C| \leq k$.
2. Verificar que cada elemento de $C \in A$
2. Para cada subconjunto B_i , verificar que existe al menos un elemento perteneciente simultáneamente a C y a B_i .

Validador

```
1 def es_Hitting_set(A,B,C,k):  
2     if len(C)>k:  
3         return False  
4     for elemento in C:  
5         if elemento not in A:  
6             return False  
7     for subset in B:  
8         if not any(x in C for x in subset):  
9             return False  
10    return True
```

La primera verificación puede realizarse en tiempo lineal respecto del tamaño de C . Para la segunda, si trabajo con sets la verificación de pertenencia se realiza en tiempo constante, recorro cada elemento de C y verifico si se encuentra en A me cuesta $O(n)$ en el peor caso, si C y A tienen los mismos elementos. Para la tercera basta recorrer cada conjunto B_i y comprobar si alguno de sus elementos pertenece a C . En el peor caso, esto requiere tiempo $O(m \cdot n)$, que es polinomial en el tamaño de la entrada.

Dado que existe un certificado de tamaño polinomial cuya validez puede verificarse en tiempo polinomial, concluimos que: $\text{Hitting Set} \in \text{NP}$.

2. Demostración de que Hitting Set es NP-Completo

Para demostrar que *Hitting Set* es NP-Completo debemos probar dos propiedades:

1. *Hitting Set* pertenece a NP.
2. *Hitting Set* es NP-Completo.

La primera propiedad fue demostrada en la sección anterior. Resta demostrar que es NP-Hard.

2.1. Reducción desde Vertex Cover

Utilizaremos una reducción polinomial desde el problema *Vertex Cover*, conocido por ser NP-Completo.

Recordamos la definición de *Vertex Cover*:

Dado un grafo $G = (V, E)$ y un entero k , determinar si existe un conjunto de vértices $S \subseteq V$ tal que $|S| \leq k$ y toda arista de E tenga al menos uno de sus extremos en S .

Sea una instancia arbitraria de *Vertex Cover* formada por un grafo $G = (V, E)$ y un entero k . Construimos una instancia de *Hitting Set* de la siguiente manera:

- El conjunto base será $A = V$.
- Para cada arista $e = (u, v) \in E$, definimos un subconjunto: $B_e = \{u, v\}$.
- Conservamos el mismo valor de k .

La construcción es claramente polinomial, ya que para cada arista se crea un conjunto de tamaño 2.

Debemos demostrar que la instancia de *Vertex Cover* tiene solución si y solo si la instancia construida de *Hitting Set* tiene solución. Supongamos que existe un vertex cover $S \subseteq V$ con $|S| \leq k$.

Como S cubre todas las aristas, para cada arista (u, v) se cumple que al menos uno de sus extremos pertenece a S .

Dado que el conjunto asociado a dicha arista es $B_e = \{u, v\}$, resulta que: $S \cap B_e \neq \emptyset$.

Por lo tanto, S intersecta todos los conjuntos B_e y constituye un hitting set de tamaño a lo sumo k .

Supongamos ahora que existe un hitting set $C \subseteq A$ con $|C| \leq k$.

Como C intersecta cada conjunto $B_e = \{u, v\}$, para toda arista (u, v) se cumple que al menos uno de sus extremos pertenece a C .

Esto implica que todas las aristas del grafo están cubiertas por algún vértice de C , por lo que C es un vertex cover de tamaño a lo sumo k .

Hemos demostrado una reducción polinomial: *Vertex Cover* $\leq_p$ *Hitting Set*. Dado que *Vertex Cover* es NP-Completo, se concluye que *Hitting Set* es NP-Completo.

3. Algoritmo y Analisis de complejidad

3.1. Algoritmo de Backtracking

El algoritmo propuesto recibe la lista de jugadores (A/Universo) y los posibles subconjuntos a armar en base a esos jugadores (B/subconjuntos). En base a esto calculamos recursivamente el menor equipo posible que contenga al menos uno de los jugadores de cada equipo (B).

Implementamos las siguientes podas a las ramas de todas las soluciones posibles:

Cota superior: en base a un Hitting Set valido de tamaño reducido (solucion optima) podemos utilizarlo como una cota superior restrictiva que nos permite descartar ramas de caminos suboptimos sin tener la necesidad de recorrerlos completamente hasta las hojas.

Cobertura Previa: si el subconjunto parcial ya satisface la restricción actual, es decir si tenemos un elemento en el subconjunto parcial que es parte de la restricción actual seguimos hacia la próxima restricción sin la necesidad de explorar todas las combinaciones/ramificaciones posibles en base a todos los elementos de la restricción. Es decir, el algoritmo avanza en la profundidad del árbol de manera directa sin necesidad de abrir nuevos bucles de ramificación.

3.2. Pseudocódigo del algoritmo

```
1
2 def _BThittingSetOptimo(A, sets, indice, parcial, set_p, optimo):
3
4     if len(parcial) >= len(optimo):
5         return optimo
6     if len(set_p) == indice:
7         return parcial[:]
8     b = sets[indice]
9     if any(x in set_p for x in b):
10         return _BThittingSetOptimo(A, sets, indice+1, parcial, set_p, optimo)
11     for x in b:
12         if x not in set_p:
13             parcial.append(x)
14             set_p.add(x)
15             optimo = _BThittingSetOptimo(A, sets, indice+1, parcial, set_p, optimo)
16             parcial.pop()
17             set_p.remove(x)
18     return optimo
19
20 def hittingSetOptimo(jugadores, equipos):
21
22     indice_inicial = 0
23     parcial_inicial = []
24     optimo_inicial = list(jugadores)
25
26     return _BThittingSetOptimo(jugadores, equipos, indice_inicial, parcial_inicial,
27                               set(), optimo_inicial)
```

3.3. Análisis de Complejidad

3.3.1. Complejidad del algoritmo de Backtracking

Tenemos dos variables A (jugadores) de n elementos y B (equipos) de m subconjuntos:

En el peor caso recorremos los m subconjuntos y todas sus ramificaciones posibles. A continuación se detalla el análisis asintótico de este comportamiento:

Complejidad Temporal:

N =: El universo total de jugadores disponibles.

M =: El número total de conjuntos (equipos) que se deben *hittear*.

K =: El tamaño máximo de un equipo individual (es decir, $\max(\text{len}(b))$ para todo $b \in \text{equipos}$), donde $K \leq N$.

El algoritmo avanza de forma lineal sobre los equipos usando el parámetro *indice* (que va de 0 a M). Por cada equipo, si ese equipo ya está cubierto por el *hitting set* entonces continuo mi recursión hacia el próximo equipo. En caso contrario, recorro el equipo y por cada jugador no perteneciente al *hitting set* realizo una llamada recursiva incluyéndolo.

En el peor de los casos (ningún jugador del equipo pertenece al *Hitting Set*), se realizan tantas llamadas recursivas como elementos tenga el equipo. Es decir, hasta K ramificaciones.

En el peor escenario teórico (donde los subconjuntos son disjuntos y las podas no logran actuar tempranamente), el árbol tiene una profundidad de M niveles, y cada nivel se puede ramificar en K nuevos caminos. El número máximo de nodos visitados en el árbol será de orden:

$$\mathcal{O}(K^M)$$

Análisis del costo de las Operaciones por subconjunto:

En cada llamada de la función `_BThittingSetOptimo`, ocurren las siguientes operaciones:

- **Casos base y podas:** `len(parcial) >= len(optimo)` y `len(sets) == indice`. Son comparaciones $\mathcal{O}(1)$.
- **Verificación de cobertura:** `any(x in set_p for x in b)`. Como es un *set* cuesta $\mathcal{O}(1)$. Iterar sobre los elementos del equipo actual b toma $\mathcal{O}(K)$ en el peor de los casos.
- **Bucle For:** `for x in b:`. Itera un máximo de K veces realizando operaciones $\mathcal{O}(1)$.

Por lo tanto, el costo de procesamiento propio en cada subconjunto del árbol de recursión es $\mathcal{O}(K)$.

Complejidad Temporal Total:

Multiplicando el número total de subconjuntos por el costo de procesamiento dentro de cada uno, obtenemos la complejidad total en el peor de los casos:

$$\text{Complejidad Temporal} = \mathcal{O}(K \cdot K^M) = \mathcal{O}(K^{M+1})$$

3.4. Análisis tiempos

3.4.1. Variación de N con M fijo

Sets Disjuntos: Este escenario representa el caso frontera del problema, diseñado específicamente para anular la efectividad de las podas implementadas. Al no existir intersección de elementos entre las restricciones, el algoritmo se ve forzado a explorar la totalidad del árbol de decisiones para evaluar las combinaciones posibles de subconjuntos.

Para construir estas instancias, se distribuyen los N jugadores en M sets estrictamente disjuntos. Al ser la intersección entre cualquier par de conjuntos igual al conjunto vacío ($B_i \cap B_j = \emptyset, \forall i \neq j$), la condición de cobertura previa nunca se cumple en las primeras etapas de la recursión y la cota superior no logra actuar tempranamente.

Este comportamiento acentúa la naturaleza intrínseca del problema NP-completo, reflejando empíricamente un orden de complejidad exponencial respecto al tamaño del universo de jugadores (N).

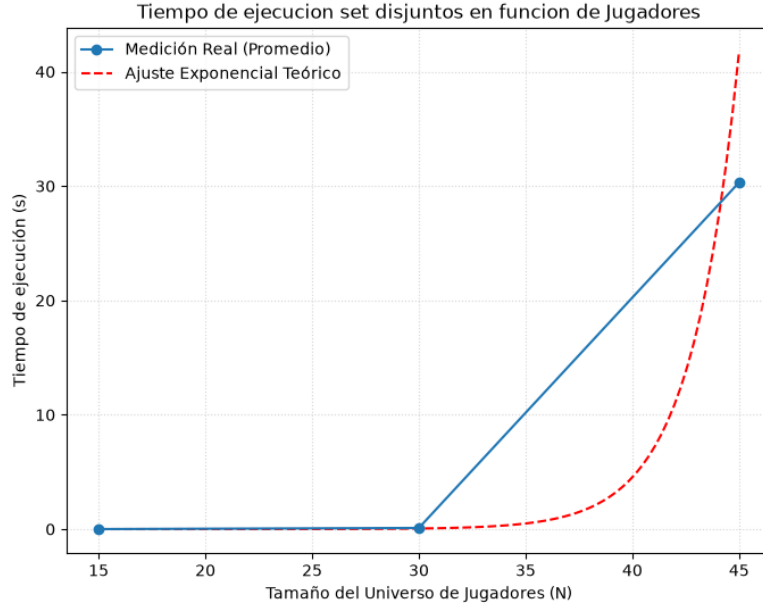


Figura 1: Tiempo de ejecución en función de la cantidad de jugadores N con M fijo.

El modelo de ajuste teórico correspondiente, calculado mediante el método de cuadrados mínimos, responde a la siguiente función:

$$T(n) = c_1 \cdot (c_2)^n$$

Donde c_1 representa la constante de proporcionalidad ligada al overhead de la máquina y las operaciones base, mientras que c_2 determina la tasa de ramificación real que experimenta el árbol de exploración de Backtracking.

Sets No Disjuntos: A diferencia del escenario disjunto, la utilización de instancias con sets no disjuntos representa un caso promedio o favorable, donde los conjuntos de restricciones comparten una cantidad significativa de elementos comunes ($B_i \cap B_j \neq \emptyset$ para diversos pares i, j). Este solapamiento estructural provee un entorno óptimo para la activación de las reglas de poda implementadas.

En primer lugar, la alta densidad de elementos repetidos incrementa drásticamente la efectividad de la poda por cobertura previa. Durante la exploración del árbol, la condición que evalúa si el subconjunto parcial ya satisface la restricción actual tiende a dar verdadero con frecuencia, permitiendo al algoritmo avanzar en la profundidad del árbol de manera directa sin necesidad de abrir nuevos bucles de ramificación.

En segundo lugar, la interconectividad de los conjuntos facilita el hallazgo muy temprano de un Hitting Set válido de tamaño reducido (óptimo global). Una vez establecida esta cota superior restrictiva, la poda por cota superior intercepta masivamente los caminos subóptimos, descartando ramas enteras mucho antes de llegar a las hojas del árbol.

Como consecuencia de este doble mecanismo de filtrado, el espacio de búsqueda efectivamente explorado se contrae de forma drástica. La curva de tiempo experimenta un comportamiento fuertemente linealizado respecto a N , desvinculándose de la tendencia exponencial teórica del peor caso.

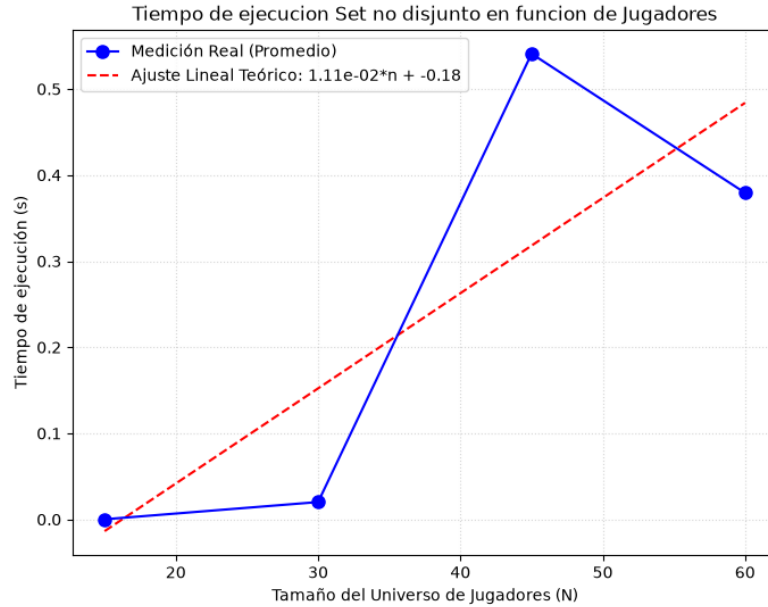


Figura 2: Tiempo de ejecución en función de la cantidad de jugadores N con M fijo.

El modelo de ajuste teórico correspondiente, calculado mediante el método de cuadrados mínimos, responde a la siguiente función:

$$T(n) = c_1 \cdot n + c_2$$

Donde c_1 representa la constante de proporcionalidad ligada al overhead de la máquina y las operaciones base, mientras que c_2 determina la tasa de ramificación real que experimenta el árbol de exploración de Backtracking.

3.4.2. Variación de M con N fija

En este escenario se observa un fenómeno del tiempo de ejecución de crecimiento al principio, pero a medida que aumenta el número de restricciones, M , el crecimiento se frena y se estabiliza. En las instancias donde $M < N$, el incremento de restricciones añade complejidad al árbol al forzar la búsqueda de nuevos elementos de cobertura. Sin embargo, una vez alcanzado el umbral donde $M \geq N$, la alta redundancia de los conjuntos no disjuntos estabiliza el tamaño del Hitting Set óptimo global. Esto desencadena una activación prematura y masiva de la poda por cota superior, cortando la expansión del árbol combinatorio y provocando que el tiempo de cómputo se amesete de forma asintótica.

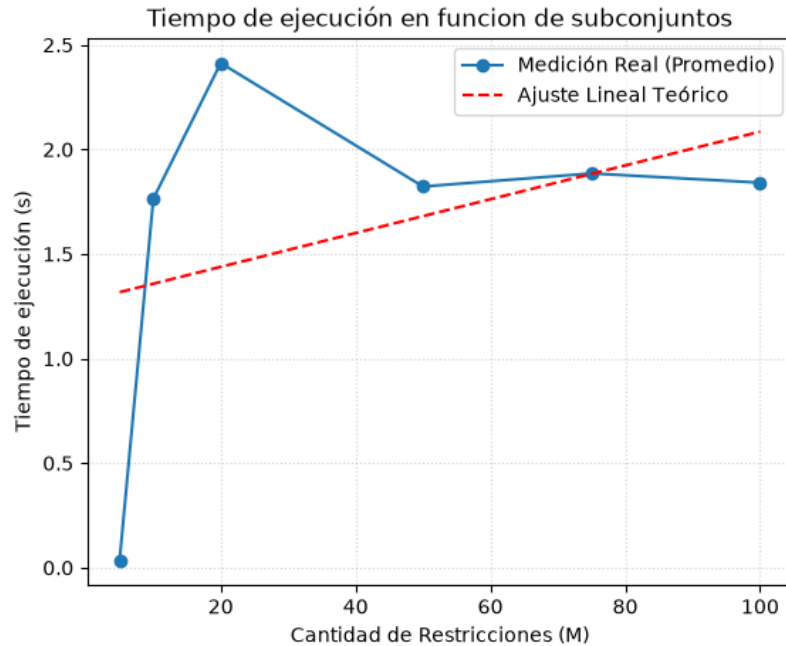


Figura 3: Tiempo de ejecución en función de la cantidad de restricciones M para un universo N fijo.

El modelo de ajuste teórico correspondiente, calculado mediante el método de cuadrados mínimos, responde a la siguiente función:

$$T(m) = c_1 \cdot m + c_2$$

4. Conclusión

En este trabajo se estudió el problema *Hitting Set*, demostrando formalmente que pertenece a la clase NP y que es NP-Completo mediante una reducción polinomial desde *Vertex Cover*. Esta caracterización teórica permite comprender la dificultad intrínseca del problema y justifica la necesidad de recurrir a técnicas de búsqueda exhaustiva para obtener soluciones óptimas.

Para resolver el problema se implementó un algoritmo de *backtracking* complementado con estrategias de poda por cobertura previa y por cota superior. Estas optimizaciones permitieron reducir significativamente el espacio de búsqueda explorado en numerosos casos, conservando la garantía de obtener siempre una solución óptima.

Los experimentos realizados evidencian la fuerte influencia que tiene la estructura de las instancias sobre el rendimiento del algoritmo. En los casos con conjuntos disjuntos, donde las podas resultan poco efectivas, los tiempos de ejecución reflejan el crecimiento exponencial esperado para un problema NP-Completo. En cambio, cuando existe un alto grado de solapamiento entre los conjuntos, las podas logran eliminar gran parte de las ramas del árbol de búsqueda, produciendo tiempos considerablemente menores y un comportamiento cercano al lineal en la práctica.

En conclusión, los resultados obtenidos muestran tanto la correctitud del algoritmo implementado como las limitaciones inherentes de los métodos exactos para resolver problemas NP-Completo. A su vez, permiten apreciar cómo la utilización de podas adecuadas puede mejorar sustancialmente el desempeño práctico sin sacrificar la optimalidad de la solución encontrada.